Estructuras de Datos

2026
Primer Cuatrimestre

UNS UNIVERSIDAD NACIONAL DEL SUR

Profesor

Axel J. Soto (axel.soto@cs.uns.edu.ar)

Ingeniero en Sistemas de Computación

Doctor en Ciencias de la Computación





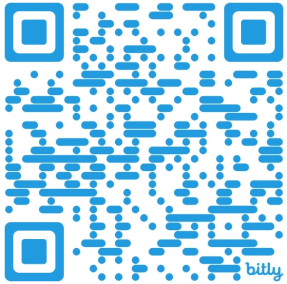


Profesor

Axel J. Soto (axel.soto@cs.uns.edu.ar)

Temas de Investigación

- Inteligencia Artificial - Machine Learning
- Procesamiento de lenguaje natural
- Químicainformática



https://bit.ly/axelsotoICIC

Profesor

Axel J. Soto (axel.soto@cs.uns.edu.ar)

Docencia

- Introducción a la Ingeniería de Software
- Estructuras de Datos
- Ciencia de Datos

Cursado



Lab004
DCIC

Lab004
DCIC

Cambio de comisión

Es posible pero tienen que brindar una razón válida

En Moodle podrán encontrar el link al formulario

Condiciones de Cursado

Dos parciales + un único recuperatorio al final del cuatrimestre

Un **proyecto** al final del cuatrimestre + su recuperatorio

Condiciones de Cursado

Dos parciales + un único recuperatorio al final del cuatrimestre

Un proyecto al final del cuatrimestre + su recuperatorio

La materia es promocionable



Promoción de la Materia

Aprobar ambos parciales con **cualquier combinación de A y/o B**; también se acepta **una C en el primer parcial y una A en el segundo** para promocionar.

Tener **aprobado el proyecto** de la materia en alguna de sus instancias (entrega o re-entrega);

Rendir un **examen de promoción** en la última semana del cuatrimestre.

Motivación

¿Qué les parece que tiene que tener un buen restaurante?

Un buen restaurante tiene tres pilares

Motivación

Presentación - ambiente





Motivación

El chef, la receta





Motivación

El orden de la cocina/alimentos





Sistemas de software



UI



Lógica



Estructuras de datos

Motivación - Estructuras de Datos

Desde un punto de vista conceptual, todo lo que mantiene unido a ciertos datos conforma una estructura de datos

Motivación - Estructuras de Datos

Desde un punto de vista conceptual, todo lo que mantiene unido a ciertos datos conforma una estructura de datos

Una fecha es una estructura de datos

11 JUL 1999

Motivación - Estructuras de Datos

Desde un punto de vista conceptual, todo lo que mantiene unido a ciertos datos conforma una estructura de datos

Una fecha es una estructura de datos

11 JUL 1999

FECHA = Día + Mes + Año

Motivación - Estructuras de Datos

Desde un punto de vista conceptual, todo lo que mantiene unido a ciertos datos conforma una estructura de datos



Un DNI es una estructura de datos

Motivación - Estructuras de Datos

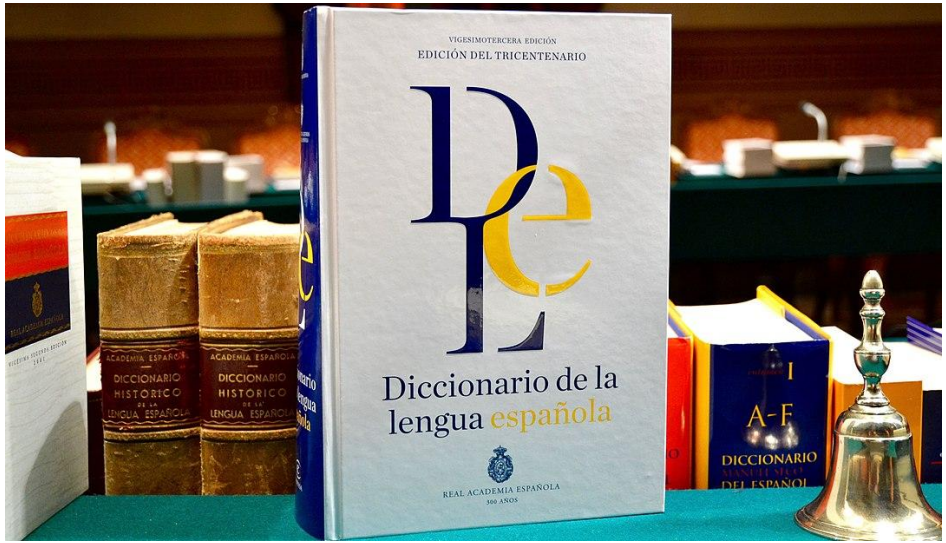
Desde un punto de vista conceptual, todo lo que mantiene unido a ciertos datos conforma una estructura de datos



Una estructura de datos puede contener otras estructuras de datos

Motivación - Estructuras de Datos

Desde un punto de vista conceptual, todo lo que mantiene unido a ciertos datos conforma una estructura de datos



Motivación - Estructuras de Datos

Una estructura se va a definir en función de lo que queramos almacenar y de lo que queramos hacer con lo almacenado

Motivación - Estructuras de Datos

Hay un gran abanico de estructuras definidas que favorecen ciertas características sobre otras

Motivación - Estructuras de Datos

¿Cómo podemos evaluar la calidad de las estructuras?

Motivación - Estructuras de Datos

~~¿Cómo podemos evaluar la calidad de las estructuras?~~

¿Cómo podemos evaluar la calidad de la programación?

Factores de calidad del software

- Factores externos:
  - Cualidad del software que puede ser detectada por un usuario (sin conocer el código fuente).
- Factores internos de calidad:
  - Cualidad del software que puede ser percibida por el programador (conociendo el código fuente)

Motivación - Factores de Calidad

Correctitud

Robustez

Extensibilidad

Portabilidad

Reusabilidad

Eficiencia

Facilidad de Uso

Factores de calidad

- Correctitud: El programa respeta la especificación (para una entrada dada produce la salida esperada).
- Robustez: El programa es capaz de manejar entradas no contempladas por la especificación (se espera que se degrade “graciosamente” o “gracefully”).
  - Ej.: Recibe un string cuando espera un entero.
  - Factor crítico en aplicaciones industriales.
- Extensibilidad: Capacidad de agregarle funcionalidad al software

Factores de calidad

- Portabilidad: Habilidad del soft de correr con mínimo cambio en otro hard o sistema operativo
- Reusabilidad: Capacidad de usar el código en la construcción de otros programas
- Eficiencia: Capacidad del soft de requerir pocos recursos de cómputo (tiempo de CPU y memoria)
- Facilidad de uso: Capacidad del soft para ser usado por usuarios con distinta habilidad

Motivación - ¿Cuánto tardan las cosas en resolverse?

A lo largo de la materia, vamos a introducir una nueva forma de “evaluar” la calidad de nuestro código. La misma nos hará pensar en el tiempo que tardan nuestros algoritmos en ejecutarse.

Motivación - ¿Cuánto tardas las cosas en resolverse?

A lo largo de la materia, vamos a introducir una nueva forma de “evaluar” la calidad de nuestro código. La misma nos hará pensar en el tiempo que tardan nuestros algoritmos en ejecutarse.

¿Cuántos alumnos hay

en el aula?



Motivación - ¿Cuánto tardas las cosas en resolverse?

A lo largo de la materia, vamos a introducir una nueva forma de “evaluar” la calidad de nuestro código. La misma nos hará pensar en el tiempo que tardan nuestros algoritmos en ejecutarse.

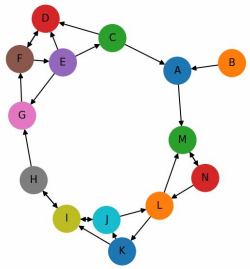
¿Cuántos alumnos hay

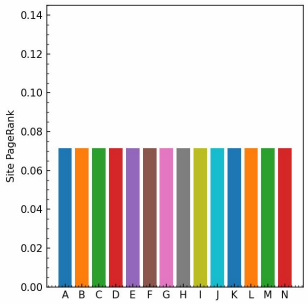
en el aula?

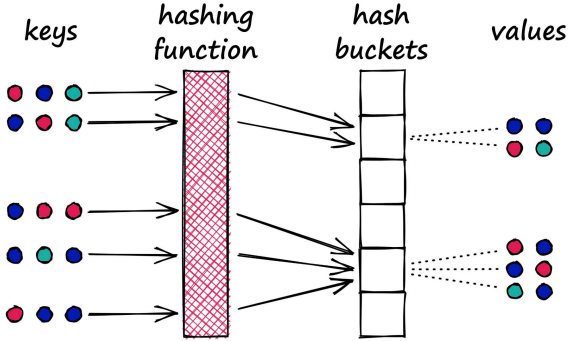
ESTRUCTURAS DE DATOS							
Comisión	Sede	Período Lectivo	Año Académico	Cant. insc.	Cant. insc. acep.	Cant. insc. pend.	Restricciones
DCIC-7655-1	BAHIA BLANCA	Primer Cuatrimestre	2026	89	0	89	Rango Apellidos
DCIC-7655-2	BAHIA BLANCA	Primer Cuatrimestre	2026	69	0	69	Rango Apellidos

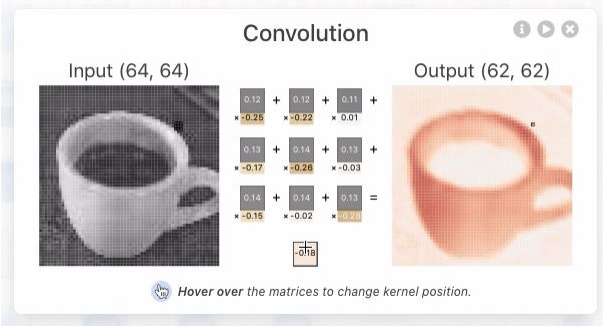
Motivación

El día a día de un programador está compuesto de usos y definiciones de estructuras de datos









Motivación

Ahora bien...

Motivación



Motivación



Motivación

Muchas veces, en el entrenamiento para una profesión, hacemos cosas que nunca vamos hacer en la profesión misma. Pero el entrenamiento aumenta nuestras posibilidades de éxito en el desempeño de la profesión

Motivación

En esta materia van a entrenar para la profesión de programadores y para eso harán cosas que en la vida profesional de programadores quizás nunca hagan. Pero las haremos para que luego puedan ser mejores programadores

BREAK

Principios de diseño OO

- Abstracción:
  - Ocultar detalles irrelevantes para aumentar entendimiento y claridad
  - Ej. tipo de dato abstracto (TDA), tipo definido en términos de operaciones
- Encapsulamiento y ocultamiento de información
  - Relacionado con la abstracción: no revelar detalles internos de implementación
  - Ej. no declarar atributos privados cuando deben ser públicos
- Modularidad:
  - Principio organizacional que busca que los componentes de software se deben separar en unidades funcionales (módulos)
  - Ejemplo: si necesito representar personas y aulas, probablemente deba escribir dos clases distintas y no una sola para las dos.

Conceptos de POO

Clases, Métodos y Herencia

Polimorfismo

Vinculación Dinámica de Código

Repaso: clases

```
public class Persona {
        public String nombre;       // Debe ser public, protected o private? Cuál es la diferencia?
        private float peso;
        public Persona(String n, float p) {
                nombre = n; peso = p;
        }
        public float getPeso() { return peso; }
        public void setPeso( float peso ) {
                this.peso = peso;           // Por qué necesito this acá?
        }
        public String toString() { return nombre + "-" + peso; }
}
```

Simplificamos la clase

```
public class Persona {
   private String nombre;
   public Persona(String nombre) {
      this.nombre = nombre;
   }
   // método público para acceder al atributo
   public String getNombre() {
      return nombre;
   }
}
```

¿Qué les parece esto?

Aplicamos herencia

```java
public class Persona {
    private String nombre;
    public Persona(String nombre) {
        this.nombre = nombre;
    }
    // método público para acceder al atributo
    public String getNombre() {
        return nombre;
    }
}
```

```java
class Estudiante extends Persona {
    public Estudiante(String nombre) {
        super(nombre);
    }
    public void presentarse() {
        System.out.println("Hola, mi nombre es " + nombre);
    }
}
```

¿Qué les parece esto?
¿Compila?
¿Se hereda un atributo privado?

Aplicamos herencia (2)

```java
public class Persona {
    private String nombre;
    public Persona(String nombre) {
        this.nombre = nombre;
    }
    // método público para acceder al atributo
    public String getNombre() {
        return nombre;
    }
}
```

```java
class Estudiante extends Persona {
    public Estudiante(String nombre) {
        super(nombre);
    }
    public void presentarse() {
        System.out.println("Hola, mi nombre es " + getNombre());
    }
}
```

Polimorfismo (dinámico)

Un mismo método puede producir comportamientos diferentes dependiendo del tipo de objeto que lo recibe

Es decir que podemos tratar distintos tipos de objetos como si fueran del mismo tipo, pero cada uno ejecuta su propia versión del código

Polimorfismo

```java
public class Auto {

    public void mostrarSlogan() {
        System.out.println("Slogan");
    }
}
```

```java
public class Main {

    public static void main(String[] args) {

        Auto a1 = new Mercedes();
        Auto a2 = new Ford();

        a1.mostrarSlogan();
        a2.mostrarSlogan();
    }
}
```



Vinculación dinámica de código!

```java
class Mercedes extends Auto {
    @Override
    public void mostrarSlogan() {
        System.out.println("Lo mejor o nada");
    }
}

class Ford extends Auto {
    @Override
    public void mostrarSlogan() {
        System.out.println("En Ford usted puede confiar");
    }
}
```

Conceptos de POO

Clases y Métodos Abstractos

Método y clase abstracto

- Es un método sin implementación / cuerpo
- Indicado con la palabra clave `abstract` en Java
- Intuitivamente:
  - No se quiere/puede definir la implementación de este método
  - Pero se sabe que el método debe existir (con su implementación)
- En el ejemplo anterior slogan de auto podría no estar implementado
- Una clase con al menos un método abstracto define una clase abstracta
- Una clase abstracta no puede tener instancias (objetos)
  - Aun cuando todos sus métodos están implementados

Conceptos de POO

Concepto Nuevo: Interfaz

Interfaz (*interface*)

- Una interfaz es una colección de declaraciones de métodos
- Intuitivamente:
  - Una clase solemos pensarlo por lo que debe ser/tener (ej. alumno, auto)
  - Una interfaz se piensa por lo que debe hacer

```
public interface Vendible {
        public String descripcion();
        public int precioLista();
}
```

```
public interface Transportable {
        public int peso();
        public boolean peligroso();
}
```

Interfaz

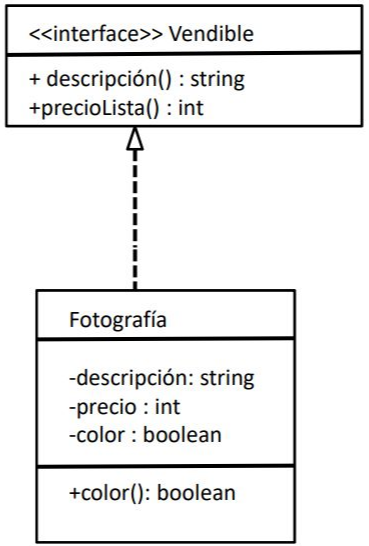
- Una clase que implementa una interfaz debe implementar todos los métodos provistos por la interfaz.

```
public class Fotografia implements Vendible {
  private String descripcion;
  private int precioLista;
  private boolean color;
  public Fotografia(String desc, int precioLis, boolean col) {
     descripcion = desc;  precioLista = precioLis; color = col;
  }
  @Override
  public String descripcion() {  return descripcion;  }
  @Override
  public int precioLista() { return precioLista; }
  public boolean color() { return color; }
}
```

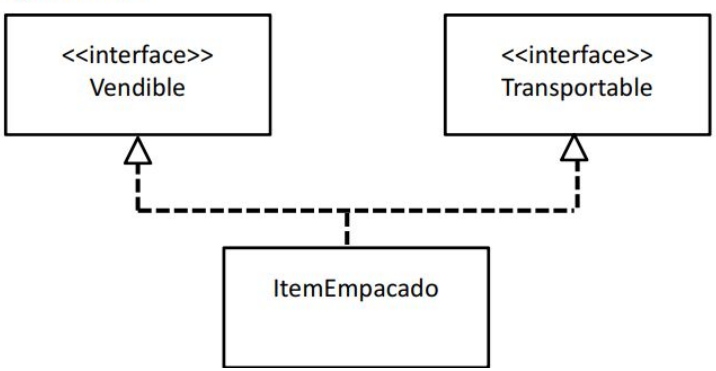
Notar la palabra *<<interface>>* para denotar una interfaz.

La flecha punteada se lee "implementa".

Modificadores de acceso:
+ significa *público*
- significa *privado*
significa *protected*

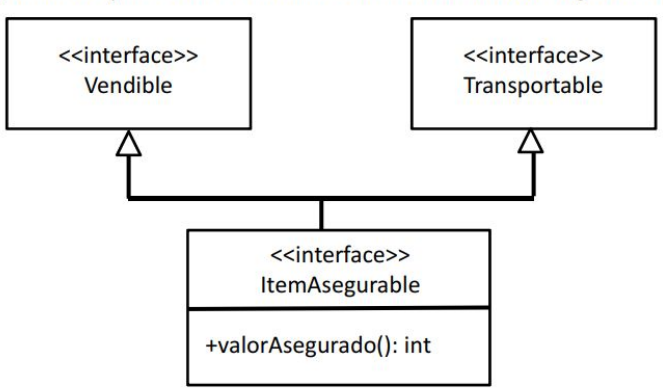


Implementación de más de una interfaz




```
public class ItemEmpacado implements Vendible, Transportable {
        //Atributos
        //Constructor
        //Implementación de todos los métodos de ambas interfaces
        //Implementación de métodos adicionales
        …
}
```

Herencia de interfaces




```
public interface ItemAsegurable extends Vendible, Transportable {
        public int valorAsegurado();
}
```

Interfaz

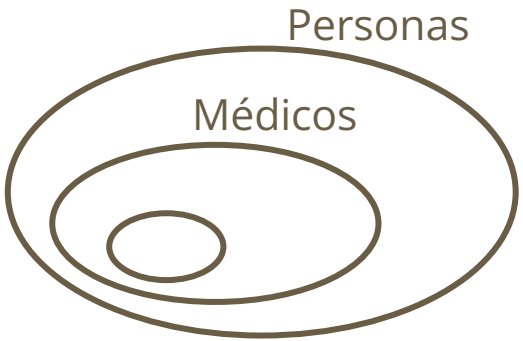
Adelantándonos...
¿Por qué son relevantes las interfaces para la materia?

- Vamos a definir estructuras de datos no por lo que son sino por lo que hacen
    - Es decir, las operaciones que la estructura de datos tiene que ser capaz de implementar

Interfaces vs clases abstractas

- Una clase abstracta es una clase que tiene **por lo menos un método abstracto** (que no se implementa) y se señala con la keyword *abstract*
- Una interfaz sería parecida a una clase abstracta donde **todos sus métodos son abstractos**.
- Las interfaces están pensadas para dar soporte a la resolución de métodos en tiempo de ejecución igual que las clases abstractas (polimorfismo).
- Las clases abstractas están pensadas para un modelo de herencia simple (relación es-un)
- Las interfaces definen una relación puede-hacer
  - Dos clases no relacionadas pueden implementar la misma interfaz.
  - Ejemplo: Persona puede ser Transportable y Reloj también

Repaso reglas de asignación en POO



Se puede asignar directamente variables de subclases en variables de superclases:

Ejemplo: Si Medico es subclase de Persona:

```
Persona p;
Medico m = new Medico("House");
p = m; // Legal porque un médico es una persona
Medico m2 = p; // Ilegal porque a pesar de que p apunta a un objeto de tipo
               // médico no se puede asegurar que una persona sea un médico.
```

Solución: casting

```
Medico m2 = (Medico) p;
```

- Si p apunta a un Medico o un objeto de alguna de sus subclases (Cirujano, Pediatra, ...) → OK,
- Sino → error (ClassCastException).

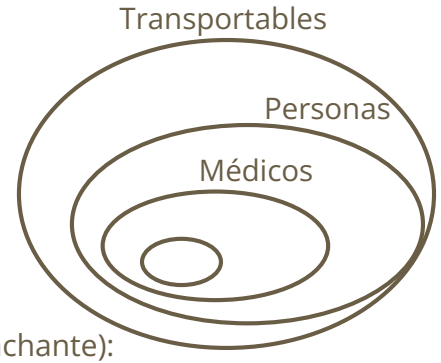
Estas reglas se extienden naturalmente a interfaces...

Reglas para asignar en forma válida

Supongamos que *a* es de tipo *U* y *b* de tipo *T*. Es válido asignar cuando:

- "*a*" y "*b*" son del mismo tipo de clase o interfaz (i.e. *T* = *U*).
- Cuando *T* se convierte en un tipo *U* más amplio, se produce una conversión widening (ensanchante):
  - T y U son nombres de clases y U es superclase de T
  - T y U son nombres de interfaces y U es superinterfaz de T
  - T es una clase que implementa la interfaz U
  - (T es una subclase de una clase que implementa una interfaz que es subinterfaz de U).
- Son todas verificables por el compilador.

Reglas para asignar en forma válida



Supongamos que *a* es de tipo *U* y *b* de tipo *T*. Es válido asignar cuando:

- "*a*" y "*b*" son del mismo tipo de clase o interfaz (i.e. T = U).
- Cuando *T* se convierte en un tipo *U* más amplio, se produce una conversión widening (ensanchante):
  - T y U son nombres de clases y U es superclase de T
  - T y U son nombres de interfaces y U es superinterfaz de T
  - T es una clase que implementa la interfaz U
  - (T es una subclase de una clase que implementa una interfaz que es subinterfaz de U).
- Son todas verificables por el compilador.

Ejemplo: Si la clase Medico es subclase de Persona:

```
Medico s = new Medico("Dr Strange");
Persona h = s;
```

Con h solo puedo usar los métodos de la clase Persona (a pesar de que h referencia a un Médico en ejecución).

Ejemplo: Si la clase Persona implementa la interfaz Transportable:

```
Transportable t = s;
```

Solo puedo ejecutar t.peligroso() y t.peso() porque t es Transportable (a pesar de que t referencia en ejecución a un médico).

Bibliografía

M.T. Goodrich & R. Tamassia. Data Structures and Algorithms in Java. Fourth Edition. John Wiley & Sons, 2006.

M.T. Goodrich, R. Tamassia, Michael Goldwasser. Data Structures and Algorithms in Java. Sixth Edition. John Wiley & Sons, 2014.

Conceptos de POO

Concepto Viejo: Genericidad por Herencia

Concepto Nuevo: Genericidad Paramétrica

Genericidad

Problema:

- Quiero programar una clase Colección una única vez y luego utilizarla para programar una colección de enteros, una colección de caracteres, una colección de personas, etc.

¿Cómo hago esto?

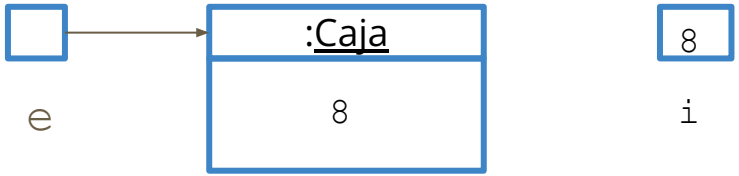
- 1er requerimiento: Los valores de los tipos básicos no se almacenan como objetos y necesito tratar a todos los tipos de datos como objetos.
- 2do requerimiento: Quiero reusar la mayor cantidad de código posible

Manejar tipos básicos como objetos: Encapsular un entero en un objeto

```
public class Caja {
      private int valor;
      public Caja(int x) { valor = x; }
      public int getValor() { return valor; }
      public void setValor( int nuevo_valor ) { valor = nuevo_valor; }
}
```

Entonces para usar estos enteros habría que hacer algo así:

```
Caja e = new Caja(8);
int i = e.getValor();
```



¿Cuál es el problema con este enfoque?

Necesitamos una caja para enteros, otra para float otra para Personas

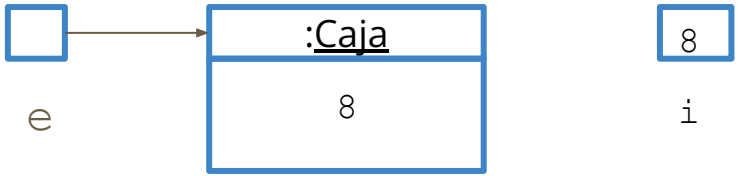
Clases wrapper

- Las clases wrapper existentes:
  - Byte para byte,
  - Short para short,
  - Integer para int,
  - Long para long,
  - Boolean para boolean,
  - Float para float,
  - Double para double
  - Character para char

- Las clases wrapper se usan como cualquier otra:

```
Integer e = new Integer(8);//Tienen constructor
int i = e.intValue();//Tienen getters: i contiene un 8
```

- Los objetos de clases wrapper son inmutables
  - no permiten actualizar los valores envueltos porque las clases wrapper no tienen setters

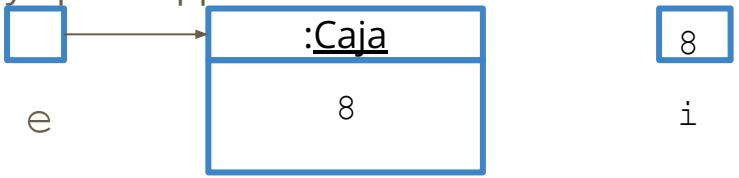


Nuevos compiladores: Autoboxing y Unboxing

- Autoboxing: Conversión automática entre tipo primitivo y tipo wrapper
  - El valor de *i* se convierte en Integer

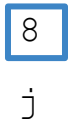
```
int i = 8;

Integer e = i;
```



- Unboxing: Conversión automática entre tipo wrapper y tipo primitivo
  - El valor en *e* se extrae y almacena en *j*

```
int j = e;
```



- Los objetos wrapper son inmutables (su valor no cambia)
- Entonces, las clases wrapper no sirven para simular pasaje de parámetros por referencia con tipos primitivos porque Java tiene pasaje de parámetros por valor (ver práctica 1)
- Recordar: Los tipos primitivos tampoco se pueden pasar por referencia

Genericidad con herencia (concepto de IPOO)

¿Cómo generalizo la idea de la caja que contenga cualquier tipo?

```
public class CajaVieja {
      private Object dato;
      public CajaVieja (Object dato) { this.dato = dato; }
      public Object getDato() { return dato; }
}
```

En un programa cliente:

```
CajaVieja cajaEntero = new CajaVieja(8);
int x = cajaEntero.getDato();
```



error de compilación

Genericidad con herencia (concepto de IPOO)

¿Cómo generalizo la idea de la caja que contenga cualquier tipo?

```
public class CajaVieja {
      private Object dato;
      public CajaVieja (Object dato) { this.dato = dato; }
      public Object getDato() { return dato; }
}
```

En un programa cliente:

```
CajaVieja cajaEntero = new CajaVieja(8);
int x = (Integer) cajaEntero.getDato();
```



```
CajaVieja cajaString = new CajaVieja("Hola");
String s = (String) cajaString.getDato();
```



```
int y = (Integer) cajaString.getDato();
```



error tiempo ejecución (ClassCastException)

Genericidad con herencia (concepto de IPOO)

¿Cómo generalizo la idea de la caja que contenga cualquier tipo?

```
public class CajaVieja {
      private Object dato;
      public CajaVieja (Object dato) { this.dato = dato; }
      public Object getDato() { return dato; }
}
```

En un programa cliente:

```
CajaVieja cajaEntero = new CajaVieja(8);
int x = (Integer) cajaEntero.getDato();
```



```
CajaVieja cajaString = new CajaVieja("Hola");
String s = (String) cajaString.getDato();
```



```
cajaEntero = cajaString;
```



compila y ejecuta (tirará error si se busca hacer getDato y castear a entero). Type unsafe

Genericidad con polimorfismo paramétrico

- Java 5.0 incluye un framework de genericidad para usar tipos de dato abstracto que evitan casting explícito
- Un tipo genérico es un tipo que no es definido en compilación sino en tiempo de ejecución
    - permite definir una clase en función de parámetros formales de tipo que abstraen los tipos de algunos datos internos de una clase
- Dada una clase genérica, vamos instanciar objetos usando parámetros actuales de tipo

Genericidad paramétrica

Actualicemos la CajaVieja de Object

```
public class Caja<E> {
      private E dato;
      public Caja(E dato) { this.dato = dato; }
      public E getDato() { return dato; }
      public void setDato( E nuevo_valor ) {
            dato = nuevo_valor; }
}
```

- E se refiere a un tipo en particular
- El constructor toma un objeto de tipo E
- La función getDato devuelve un objeto de tipo E

Para usar esta clase E debe ser reemplazado con una clase específica:

```
Caja<Persona> cp = new Caja<Persona>( p );
// cp es una caja de una persona

Caja<Float> cf = new Caja<Float>(4.5f); //
cf es una caja de un número real

Caja<String> cs = new Caja<String>("pepe");
// cs es una caja de un string
```

Comparación

```
CajaVieja cajaEntero = new CajaVieja(8);
int x = (Integer) cajaEntero.getDato();

CajaVieja cajaString = new CajaVieja("Hola");
String s = (String) cajaString.getDato();
```

```
int y = (Integer) cajaString.getDato();
```



```
cajaEntero = cajaString;
```



```
Caja<Integer> cajaInt =  new Caja<Integer>(8);

int x = cajaInt.getDato();  // No se necesita cast

Caja<String> cajaStr =  new Caja<String>("hola");

String s = cajaStr.getDato();  //No se necesita
cast
```

Las siguientes sentencias dejan de compilar:

```
String s = (String) cajaInt.getDato();

int y = (Integer) cajaStr.getDato();

cajaInt = cajaStr;
```

Los errores que antes teníamos en ejecución ahora se identifican en compilación

Genericidad con polimorfismo paramétrica
Algunas observaciones

Una clase puede tener múltiples parámetros:

```
class Cosas<A, B, C> { … }
```

Se puede hacer subclassing de clases parametrizadas:

```
class CajaEntero extends Caja<Integer> { … nuevos métodos ….}
```

Se puede extender un tipo parametrizado:

```
class CajaEspecial<E> extends Caja<E> { … nuevos métodos….}
```

La siguiente sentencia es legal:

```
Caja<String> cs = new CajaEspecial<String>(“hola”)
```

Porque caja especial es subclase de caja.

Genericidad con polimorfismo paramétrica
Algunas observaciones (2)

Una clase parametrizada es como cualquier otra clase.

Puede ser usada como tipo de parámetros formales y como tipo de retorno de funciones:

```
Caja<String> método(int i, Caja<Integer> c) { … }
```

Con la caja, puedo simular pasaje por referencia con tipos inmutables:

```
void metodo2( Caja< Integer> c ) {
    // Modifico el estado interno de c con un setter:
    c.setDato(8);
}
```

Genericidad con polimorfismo paramétrica

Problema: Programar una clase para representar un par ordenado que permita almacenar dos valores de cualquier tipo.

• La clase será genérica, se llamará Pair y estará parametrizada en términos de los tipos de las dos componentes del par.

• (Solución presentada en el libro)

```java
/* Implementa un par ordenado (clave, valor) */
public class Pair<K,V>   // K y V son parámetros formales de tipo.
{
    // Los parámetros formales de tipo se pueden usar
    // para declarar atributos y variables:
    protected K key;   // key es tipo K
    protected V value;   // value es de tipo V

    // Los parámetros formales de tipo se pueden usar como
    // tipo en un parámetro formal:
    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    // Los parámetros formales de tipo se pueden usar como tipo
    // de retorno de una función
    public K getKey() { return key; }
    public V getValue() { return value; }
    public String toString() {
        return "[" +  getKey() +
        "," + getValue() + "]";
    }
}
```

Usando la clase pair

- Crear un par de tipo (string, número real) y le almacenaremos el par (“Sergio”, 3.14):

```
Pair<String, Float> p = new Pair<String,Float>("Sergio", 3.14f);
```

- Crear un par de tipo (entero, carácter) y le almacenaremos el par (8, ‘b’):

```
Pair<Integer, Character> q = new Pair<Integer, Character>(8, 'b’);
```

- Crear un par de tipo (persona, persona) y le almacenaremos a Juan de 80 kg como primera componente y a Tito de 100kg años como segunda:

```
Pair<Persona, Persona> r = new Pair<Persona, Persona>(new Persona("Juan", 80.0f),
new Persona("Tito", 100.0f));
```

- Nota: Usar tipos wrapper en la declaración y no tipos básicos.
  - 3.14f de tipo float se convierte automáticamente en Float.

Conjunto con genericidad con polimorfismo paramétrico

- **Problema**: Implementar una colección genérica
- Para ello utilizaremos como estructura de datos subyacente un arreglo de elementos de tipo E
- Luego crearemos un conjunto de enteros con el contenido { 1, 2 }
- Luego crearemos otro conjunto de caracteres con los elementos { a, b }

- Usar una **interfaz** para especificar las operaciones en forma **independiente de las posibles implementaciones**
  - De esta forma esta colección en un tipo de dato abstracto (solo especifico operaciones)

Interfaz *coleccion*

```
//Archivo: Coleccion.java (para completar con más operaciones):
//Note que la interface es genérica, tiene un parámetro formal de tipo E
//que indica el tipo de los elementos de la colección.
public interface Coleccion<E> {
        //Permite insertar item al final de la colección.
        public void insertar(E item);
        //Permite recuperar el elemento i-esimo de la colección
        public E recuperar(int i);
}
```

Clase ColeccionConArreglo<E>

```
// Archivo fuente: ColeccionConArreglo.java
public class ColeccionConArreglo<E> implements Coleccion<E>
{
      protected int cant; // cantidad de elementos almacenados
      protected E [] datos; // arreglo de elementos
      public ColeccionConArreglo(int tamanioMaximo) {
            cant = 0; // Hay 0 elementos al crear la colección
            datos = (E []) new Object[tamanioMaximo]; // produce warning
      }
      public void insertar(E item) {
            if( cant == datos.length ) {
                  // Incrementar tamaño del arreglo datos
                  // Tarea para el lector
            }
            datos[cant++] = item;
      }

      // Precondición: i está entre 0 y la cantidad de elementos-1
      public E recuperar(int i) {
            return datos[i];
      }
}
```

¿Cómo hace el cliente para ver cuántos elementos hay en la colección?

¿Qué ocurre si el cliente no envía un “i” en rango adecuado?

Test

```
public class Test
{
	public static void main( String [] args ) {
		// Creo una colección de enteros.
		// Debo usar la clase wrapper Integer porque los parámetros actuales de tipo no pueden ser tipos básicos.
		// Note como asignamos un objeto de tipo clase  ColeccionConArreglo a una variable de tipo interfaz Colección
		Coleccion<Integer> conj_int = new ColeccionConArreglo<Integer>(5);
		conj_int.insertar( 1 ); // autoboxing de int a Integer
		conj_int.insertar( new Integer(2) ); // Alternativa redundante
		int i = conj_int.recuperar( 1 ); // Sin casting!
		System.out.println( "x: " + i ); // Imprime x: 2

		// Creo una colección de caracteres
		Coleccion<Character> conj_ch =
		new ColeccionConArreglo<Character>(5);
		conj_ch.insertar( 'a' ); // Autoboxing de char a Character
		conj_ch.insertar( 'b' );
		char c = conj_ch.recuperar( 1 );
		// Sin casting! + unboxing de Character a char ( recuperar retorna un Character que es convertido a char para hacer la
asignación).
		System.out.println( "c: " + c ); // Imprime: c : b
	}
}
```

Resumen

- **Contexto curricular**: 3era materia de programación
- **Objetivos**: Programar ED + algoritmos y evaluarlos
- **Factores de calidad**: garantizan buenas soluciones, OOP (Correctitud, Robustez, Extensibilidad, Portabilidad, Reusabilidad, Eficiencia, Facilidad de Uso)
- **Interfaces**: Generaliza la noción de clase abstracta (interface, implements, extends, herencia múltiple de interfaces)
- **Clases Wrapper**: Usar tipos básicos como objetos (Character, Integer, Float, Double, Boolean, autoboxing, unboxing)
- **Genericidad con polimorfismo paramétrico**: Mejorar reusabilidad (class Pair<A,B>, class Coleccion<E>, instanciar con Pair<Integer, Float>, Coleccion<Persona>)

THE END